408 操作系统——页面置换算法（完整复习笔记）

页面置换算法是 408 操作系统中**虚拟内存管理**的高频考点，尤其喜欢考：

1. 给定页面访问序列，计算**缺页次数**、**缺页率**。
2. 使用 FIFO、LRU、OPT 算法模拟页面置换过程。
3. 判断哪一个页面应该被淘汰。
4. 区分 FIFO、LRU、CLOCK、改进型 CLOCK。
5. 理解 Belady 异常、局部性原理、工作集与页面抖动。

其中，**FIFO、LRU、OPT 必须熟练掌握手工模拟**，CLOCK 和改进型 CLOCK 要掌握指针移动及访问位、修改位的判断。

一、页面置换的基本概念

1. 什么是页面置换?

在请求分页存储管理中：

- 进程运行时，不需要一次性将所有页面装入内存。
- CPU 访问某个页面时，如果该页面不在内存中，就会发生**缺页中断**。
- 如果内存中分配给该进程的物理块已经全部占满，就必须选择一个页面淘汰，为新页面腾出空间。

这个过程称为**页面置换**。

例如：

进程拥有 3 个物理块，依次访问页面：

$$1,\ 2,\ 3,\ 4$$

访问前 3 个页面：

$$[1, 2, 3]$$

此时内存已满。

访问页面 4 时，发现页面 4 不在内存中：

1. 发生缺页中断。
2. 根据页面置换算法选择淘汰页面。
3. 如果被淘汰页面已修改，需要先写回外存。
4. 将页面 4 调入内存。
5. 更新页表等管理信息，恢复执行。

假设淘汰页面 1：

$$[1, 2, 3] \rightarrow [4, 2, 3]$$

2. 缺页次数与缺页率

缺页次数

每次访问的页面不在内存中，就产生一次缺页。

注意：

首次访问某个页面，如果它不在内存中，也算缺页。

缺页率

$$\boxed{\text{缺页率} = \frac{\text{缺页次数}}{\text{页面访问总次数}}}$$

例如：

访问页面 20 次，其中发生 6 次缺页：

$$\text{缺页率} = \frac{6}{20} = 30\%$$

缺页次数与页面置换次数的区别

这两个概念特别容易混淆。

概念	含义
缺页次数	访问的页面不在内存中
页面置换次数	内存已满，需要淘汰旧页面
页面命中次数	访问的页面已经在内存中
缺页率	缺页次数 ÷ 总访问次数

例如：

有 3 个空物理块，访问序列为：

$$1,\ 2,\ 3,\ 4,\ 1$$

假设采用 FIFO：

- 访问 1、2、3：缺页，但不需要置换。
- 访问 4：缺页，并置换 1。
- 访问 1：缺页，并置换 2。

因此：

$$\boxed{\text{缺页次数} = 5}$$

$$\boxed{\text{页面置换次数} = 2}$$

考试速记：

> 初始物理块为空、各物理块最终都被装满的情况下：
> 页面置换次数 = 缺页次数 − 初始空闲物理块数。

前提是题目没有额外的预装入、驻留集变化等情况。

二、最佳页面置换算法 OPT

1. 基本思想

OPT（Optimal Page Replacement），最佳页面置换算法。

淘汰未来最长时间内不会再使用的页面。

如果某个页面未来再也不会使用，则优先淘汰。

OPT 是理论上缺页次数最少的算法，但需要提前知道未来的页面访问序列，因此无法在一般操作系统中实际实现。

主要用于：

- 作为理论最优标准。
- 与其他页面置换算法比较缺页次数。
- 408 考试中的模拟计算。

2. 算法步骤

当需要页面置换时：

1. 查看当前内存中的所有页面。
2. 从当前位置向后查看未来的访问序列。
3. 找出未来最晚才会访问的页面。
4. 淘汰该页面。

如果有页面未来不会再被访问，则可以直接淘汰它。

3. 例题

假设有 3 个物理块。

页面访问序列：

$$7,\ 0,\ 1,\ 2,\ 0,\ 3,\ 0,\ 4$$

使用 OPT：

访问页面	内存状态	是否缺页	淘汰页面
7	7，-，-	是	无
0	7，0，-	是	无
1	7，0，1	是	无
2	2，0，1	是	7
0	2，0，1	否	无
3	3，0，1	是	2
0	3，0，1	否	无
4	4，0，1	是	3

注意：从访问页面 2 开始，页面 7 和 1 在剩余序列中都不再出现，所以可以任选其一淘汰。这里选择 7。

随后访问页面 3 时，页面 2 和 1 都不再出现，因此也可以任选其一。

总缺页次数：

$$\boxed{6}$$

缺页率：

$$\frac{6}{8} = 75\%$$

4. 快速记忆

$$\boxed{\text{OPT：看未来，谁最晚用就淘汰谁}}$$

三、先进先出页面置换算法 FIFO

1. 基本思想

FIFO（First In First Out），先进先出页面置换算法。

淘汰最早进入内存的页面。

与队列的先进先出规则相同。

注意：

页面被访问，并不会改变它进入内存的先后顺序。

2. 算法步骤

1. 每次调入新页面，将其加入队列尾部。
2. 发生缺页且内存已满时，淘汰队首页面。
3. 将新页面加入队尾。
4. 页面命中时，不调整队列顺序。

3. 例题

物理块数：

$$m = 3$$

页面访问序列：

$$1,\ 2,\ 3,\ 1,\ 4,\ 2,\ 5$$

采用 FIFO：

访问页面	内存状态（按进入顺序）	是否缺页	淘汰页面
1	1	是	无
2	1，2	是	无
3	1，2，3	是	无
1	1，2，3	否	无
4	2，3，4	是	1
2	2，3，4	否	无
5	3，4，5	是	2

总缺页次数：

$$\boxed{5}$$

页面置换次数：

$$\boxed{2}$$

特别注意：

访问第 4 个页面时，再次访问页面 1。

虽然页面 1 刚刚被访问，但它仍然是最早进入内存的页面。

因此访问页面 4 时，仍然淘汰页面 1。

4. FIFO 的缺点

FIFO 只考虑进入内存的时间，不考虑页面的使用频率。

因此可能出现：

经常使用的页面，因为进入内存时间较早而被淘汰。

此外，FIFO 还可能出现 Belady 异常。

5. Belady 异常

正常情况下，我们可能认为：

物理块数量增加，缺页次数应该减少。

但是 FIFO 存在一种特殊现象：

$$\boxed{\text{物理块数增加，缺页次数反而增加}}$$

这种现象称为：

Belady 异常。

经典访问序列：

$$1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5$$

物理块数	FIFO 缺页次数
3	9
4	10

虽然物理块从 3 个增加到 4 个，缺页次数却从 9 次增加到 10 次。

408 必背：

- FIFO：可能发生 Belady 异常。
- LRU：不会发生 Belady 异常。
- OPT：不会发生 Belady 异常。

6. 快速记忆

$$\boxed{\text{FIFO：看过去，谁先进来就淘汰谁}}$$

四、最近最久未使用页面置换算法 LRU

1. 基本思想

LRU（Least Recently Used），最近最久未使用页面置换算法。

淘汰过去最长时间没有被访问的页面。

它利用程序访问的时间局部性：

最近访问过的页面，未来很可能再次访问。

LRU 与 FIFO 最大的区别：

- FIFO：看页面进入内存的时间。
- LRU：看页面最近一次被访问的时间。

注意：LRU 中，页面命中也会更新其最近使用情况。

2. 算法步骤

当需要置换时：

1. 查看当前内存中的所有页面。
2. 找出它们最近一次被访问的时间。
3. 淘汰距离当前时间最久的页面。

3. 例题

假设有 3 个物理块。

页面访问序列：

$$1,\ 2,\ 3,\ 1,\ 4,\ 2,\ 5$$

使用 LRU。

访问页面	内存状态（左侧最久未使用）	是否缺页	淘汰页面
1	1	是	无
2	1，2	是	无
3	1，2，3	是	无
1	2，3，1	否	无
4	3，1，4	是	2
2	1，4，2	是	3
5	4，2，5	是	1

总缺页次数：

$$\boxed{6}$$

页面置换次数：

$$\boxed{3}$$

重点分析

访问序列前四项：

$$1,\ 2,\ 3,\ 1$$

此时内存包含：

$$\{1, 2, 3\}$$

其中：

- 页面 1：刚刚访问。
- 页面 2：很久没有访问。
- 页面 3：比页面 2 更晚访问。

因此，访问页面 4 时：

$$\boxed{\text{淘汰页面 } 2}$$

而 FIFO 会淘汰页面 1。

这就是两者的核心区别。

4. LRU 的实现

LRU 可以采用：

- 计数器。
- 时间戳。
- 双向链表等数据结构。

一种典型实现是：

维护一个按最近使用时间排序的链表。

每次访问页面：

1. 如果页面已经存在，将其移动到链表尾部。
2. 如果页面不存在，将其加入链表尾部。
3. 如果需要置换，淘汰链表头部的页面。

但精确实现 LRU 会带来一定的硬件或软件管理开销。

5. 快速记忆

LRU：看过去，谁最久没用就淘汰谁

五、时钟页面置换算法 CLOCK

1. 基本思想

CLOCK 又称为：

- 时钟置换算法。
- 最近未使用算法的近似实现。
- 第二次机会算法（Second Chance）的环形实现。

CLOCK 使用一个循环链表，并为每个页面设置一个访问位 A。

访问位：

A	含义
0	最近未被访问，允许淘汰
1	最近被访问，暂时不淘汰

注意：这里的“最近”取决于访问位何时被清零，并非某个固定时间范围。

2. 算法步骤

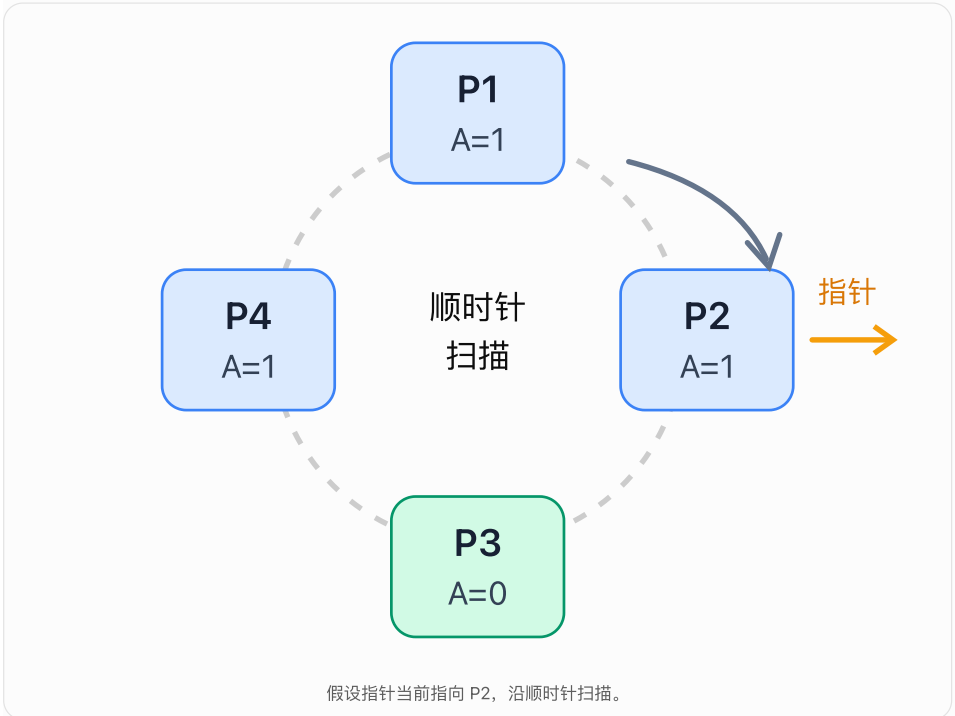
1. 页面被访问时，将访问位设置为 1。
2. 发生缺页且需要置换时，从时钟指针当前位置开始检查。
3. 如果 A=0，淘汰该页面。
4. 如果 A=1，将其修改为 0，并让指针向前移动。
5. 重复检查，直到找到 A=0 的页面。

新页面装入后，通常将其访问位置为 1，指针移动到下一位置。

具体题目如果规定其他指针或访问位初始化规则，以题目为准。

3. 示意图

假设有 4 个物理块：



假设指针当前指向 P2，沿顺时针扫描。

当前：

$$P_1(A=1),\ P_2(A=1),\ P_3(A=0),\ P_4(A=1)$$

指针指向 P2。

发生页面置换：

1. 检查 P2：A=1，修改为 0，跳过。
2. 检查 P3：A=0，淘汰 P3。
3. 新页面装入 P3 所在物理块。

因此：

$$\boxed{\text{淘汰 P3}}$$

4. CLOCK 的优缺点

优点：

- 实现相对简单。
- 不需要记录精确的页面访问时间。
- 性能通常优于简单 FIFO。

缺点：

- 只是近似 LRU。
- 可能淘汰近期访问过的页面。
- 没有充分考虑页面是否被修改。

六、改进型 CLOCK 页面置换算法

1. 基本思想

普通 CLOCK 只考虑访问位 A。

改进型 CLOCK 同时考虑：

- 访问位 A。
- 修改位 M。

因为被修改过的页面，淘汰时可能需要写回外存。

而未修改的页面，可以直接丢弃其内存副本。

因此，改进型 CLOCK 希望优先淘汰：

最近没有被访问，并且没有被修改的页面。

2. 四种页面类型

类型	A	M	含义	淘汰优先级
第一类	0	0	最近未访问、未修改	最高
第二类	0	1	最近未访问、已修改	较高
第三类	1	0	最近访问、未修改	较低
第四类	1	1	最近访问、已修改	最低

优先级：

$$\boxed{(0,0) > (0,1) > (1,0) > (1,1)}$$

这里的大于号表示“淘汰优先级更高”，并不表示严格按照该顺序在一次扫描中直接选取。

3. 改进型 CLOCK 的扫描过程

408 常见教材采用以下四轮扫描规则：

第一轮：寻找 (0,0)

从指针当前位置开始扫描一圈。

寻找：

$$\boxed{A=0,\ M=0}$$

如果找到，直接淘汰。

本轮不修改访问位 A。

第二轮：寻找 (0,1)

如果第一轮没有找到：

继续扫描一圈，寻找：

$$\boxed{A=0,\ M=1}$$

找到后直接淘汰。

本轮扫描过程中，将经过页面的访问位 A 置为 0。

第三轮：寻找 (0,0)

如果第二轮仍然没有找到：

再次扫描寻找：

$$\boxed{A=0,\ M=0}$$

由于第二轮已经将访问位清零，这一轮可能找到新的 (0,0) 页面。

第四轮：寻找 (0,1)

如果第三轮没有找到：

再次扫描寻找：

$$\boxed{A=0,\ M=1}$$

最终可以找到可淘汰页面。

注意：不同教材或具体实现可能采用不同的扫描细节。408 做题时应以题干规定的规则为准。

4. 快速记忆

$$\boxed{\text{先找没访问、没修改的页面}}$$

如果找不到，再考虑已修改的页面。

A 表示最近是否访问，M 表示是否修改。

七、页面置换算法综合比较

算法	淘汰依据	需要考虑的时间	Belady 异常
OPT	未来最晚使用	未来	不会
FIFO	最早进入内存	过去的装入顺序	可能
LRU	最近最久未使用	过去的访问顺序	不会
CLOCK	访问位 A	近期访问情况	可能
改进型 CLOCK	访问位 A、修改位 M	近期访问及修改情况	可能

特别注意：

CLOCK 是 LRU 的近似算法，但不具备精确 LRU 的全部性质，因此不能直接认为它一定不会出现 Belady 异常。

三大算法记忆口诀

OPT 看未来

谁未来最晚使用，淘汰谁。

FIFO 看进入顺序

谁最早进入内存，淘汰谁。

LRU 看访问顺序

谁过去最久没有访问，淘汰谁。

八、页面分配策略与置换策略

这一部分经常与页面置换算法结合考查。

1. 页面分配策略

操作系统需要决定给每个进程分配多少物理块。

主要分为：

- 固定分配。
- 可变分配。

固定分配

进程运行期间，分配的物理块数量保持不变。

例如：

进程 A 始终拥有 3 个物理块。

可变分配

进程运行期间，系统可以动态调整其拥有的物理块数量。

例如：

进程 A 最初拥有 3 个物理块。

发生频繁缺页后，增加到 5 个物理块。

2. 页面置换策略

分为：

- 局部置换。
- 全局置换。

局部置换

只能淘汰当前进程自己的页面。

全局置换

可以从系统中可参与置换的物理块中选择牺牲页面，包括其他进程的页面。

3. 三种常见组合

分配与置换组合	含义
固定分配、局部置换	物理块数固定，只置换自己的页面
可变分配、全局置换	物理块数可变，可以在全局范围选择页面
可变分配、局部置换	物理块数可变，但只置换自己的页面

注意：

固定分配、全局置换通常不作为标准组合，因为它与固定分配给进程的物理块数量这一约束不协调。

九、页面抖动（颠簸）

1. 什么是页面抖动?

页面抖动（Thrashing）是指：

进程频繁发生缺页，大量时间用于页面换入换出，而不是执行实际程序。

例如：

一个进程需要反复访问：

$$1, 2, 3, 4$$

但系统只给它 2 个物理块。

假设采用 FIFO，访问序列不断循环：

$$1, 2, 3, 4, 1, 2, 3, 4, \cdots$$

由于物理块不足，可能几乎每次访问都发生缺页。

系统不断执行：

$$\text{缺页} \rightarrow \text{换出} \rightarrow \text{换入}$$

导致 CPU 利用率下降，I/O 负担上升。

2. 页面抖动的原因

根本原因：

$$\boxed{\text{进程获得的物理块不足以容纳其当前活跃页面集合}}$$

常见诱因：

- 系统中并发进程过多。
- 进程物理块分配不足。
- 页面频繁换入换出。

注意：页面抖动不意味着 CPU 很忙。大量时间可能消耗在等待磁盘 I/O 上。

3. 如何解决页面抖动?

常见措施：

1. 增加进程可用物理块。

2. 降低多道程序并发程度。
3. 使用工作集模型控制驻留集。
4. 使用缺页率（PFF）控制策略。

十、工作集与驻留集

1. 驻留集

驻留集是指：

当前时刻，已经装入内存的某进程页面集合。

例如：

某进程在内存中有页面：

$$\{1,3,5\}$$

则它的驻留集为：

$$\boxed{\{1,3,5\}}$$

驻留集大小为 3。

2. 工作集

工作集（Working Set）：

某一段时间窗口内，进程实际访问过的页面集合。

设页面访问序列为：

$$1,2,3,2,1,4,2,5$$

工作集窗口大小：

$$\Delta = 4$$

考虑最后 4 次访问：

$$1,4,2,5$$

得到工作集：

$$\boxed{W=\{1,2,4,5\}}$$

工作集大小：

$$\boxed{|W|=4}$$

注意：

工作集窗口大小是访问次数，不一定等于工作集中的页面个数。

例如：

最后 4 次访问为：

$$1,2,1,2$$

则：

$$W = \{1, 2\}$$

因此：

$$|W| = 2$$

3. 工作集与驻留集的区别

概念	含义
工作集	最近一段时间实际访问过的页面
驻留集	当前真正存放在内存中的页面
工作集窗口	用于统计最近访问页面的时间或访问次数范围
工作集大小	窗口内不同页面的个数

如果驻留集长期小于工作集所需规模，可能导致频繁缺页。

4. 缺页率控制

PFF（Page Fault Frequency）根据缺页频率调整分配给进程的物理块数。

- 缺页率过高：适当增加物理块。
- 缺页率过低：可以考虑回收部分物理块。
- 系统物理块不足：可能需要暂停部分进程，降低并发程度。

十一、408 高频易错点

易错点	正确结论
首次访问页面是否缺页?	页面不在内存中就算
缺页是否一定发生置换?	不一定，可能还有空物理块
FIFO 页面命中后是否调整顺序?	不调整
LRU 页面命中后是否调整顺序?	调整最近使用顺序
OPT 看过去还是未来?	未来
LRU 看过去还是未来?	过去
FIFO 是否会出现 Belady 异常?	可能
LRU 是否会出现 Belady 异常?	不会
OPT 是否可以实际准确实现?	一般不能，需要预知未来
CLOCK 的 A=1 是否立即淘汰?	不会，先清零并跳过
改进型 CLOCK 优先淘汰什么?	A=0，M=0
被淘汰的脏页是否需要写回?	通常需要
工作集窗口大小是否等于工作集大小?	不一定
缺页中断属于什么中断?	内中断（异常）

十二、408 页面置换模拟练习

下面这道题可以分别使用 OPT、FIFO、LRU 三种算法模拟。物理块初始为空，固定分配 3 个物理块。

页面访问序列：

$$1,\ 2,\ 3,\ 1,\ 4,\ 2,\ 5$$



最终正确答案：

算法	缺页次数	页面置换次数	缺页率
OPT	5	2	71.4%
FIFO	5	2	71.4%
LRU	6	3	85.7%

这道题还说明一个重要结论：

LRU 不一定比 FIFO 的缺页次数少。 它通常能更好地利用局部性，但并不保证对每个访问序列都优于 FIFO。

十三、考前最后背诵版

如果时间紧张，我建议你优先记住下面 8 条：

1. **OPT：** 淘汰未来最长时间不再访问的页面。
2. **FIFO：** 淘汰最早进入内存的页面，命中不改变顺序。
3. **LRU：** 淘汰过去最久未访问的页面，命中要更新顺序。
4. **CLOCK：** A=1 就清零跳过，A=0 才淘汰。
5. **改进型 CLOCK：** 同时考虑访问位 A 和修改位 M，优先找 (0,0)。
6. **Belady 异常：** 增加物理块却使缺页次数增加。FIFO 可能发生，OPT、LRU 不会。
7. **缺页与置换：** 缺页不一定置换；初始空闲物理块装入页面也算缺页。
8. **页面抖动：** 物理块不足，导致频繁缺页换入换出；可通过调整驻留集、降低并发程度等方式缓解。

对于你现在的 408 冲刺阶段，我建议把 **FIFO、LRU、OPT 的模拟计算，以及 CLOCK 的访问位判断**作为优先拿分项。这几种算法的规则比较固定，只要页面访问序列不太长，通常可以通过规范列表和逐步模拟稳定得分。